

FORMER SERIES

Attention

Attention থেকে Multi-Head পর্যন্ত

Lecture

ভাষা

বাংলা + English Terms

ধরন

Deep Technical Guide

বিষয়বস্তু

- Self-Attention রিক্যাপ — কী এবং কেন?
- Self-Attention-এর সীমাবদ্ধতা ও সমস্যা
- Multi-Head Attention — মূল ধারণা ও সমাধান
- ধাপে ধাপে Matrix-level Process
- Original Transformer Paper-এর Implementation
- Dimension Reduction — কেন ৫১২ থেকে
- Visual Diagrams ও ASCII Architecture

→ Visualization Tool Demo বিশ্লেষণ

SECTION 01



সহজ ভাষায় পরিচিতি (Intuition)

💡 মূল প্রশ্ন

আমরা একটি বাক্যকে কীভাবে সংখ্যায় রূপান্তর করব, যাতে অর্থ বোঝা যায়? এবং একটি বাক্যের একাধিক অর্থ থাকলে কীভাবে সব অর্থ ধরব?

বাস্তব উদাহরণ দিয়ে বোঝা যাক

কল্পনা করো তুমি একটি গোয়েন্দা। তোমাকে একটি রহস্যময় বাক্য দেওয়া হলো:

🔍 রহস্যময় বাক্য

"The man saw the astronomer with a telescope"

এই বাক্যের দুটি অর্থ হতে পারে:

১. লোকটি টেলিস্কোপ দিয়ে জ্যোতির্বিদকে দেখল (লোকটির হাতে টেলিস্কোপ)
২. লোকটি জ্যোতির্বিদকে দেখল, যার হাতে টেলিস্কোপ ছিল (জ্যোতির্বিদের হাতে টেলিস্কোপ)

একজন সাধারণ গোয়েন্দা একটাই অর্থ ধরতে পারে। কিন্তু একজন দক্ষ গোয়েন্দার দল একসাথে দুটো দিক থেকে তদন্ত করে। **Multi-Head Attention হলো সেই দক্ষ গোয়েন্দার দল!**

● Self-Attention (একা গোয়েন্দা)

একটিই দৃষ্টিকোণ থেকে দেখে। একটি বাক্যে শুধু একটি অর্থ খোঁজে।

● Multi-Head Attention (দলগত তদন্ত)

একসাথে অনেক দৃষ্টিকোণ থেকে দেখে। একই বাক্যে একাধিক অর্থ ধরতে পারে।

আরেকটি বাস্তব উদাহরণ — Document Summarization

ধরো তুমি একটি document summarization tool বানাচ্ছ। যদি শুধু Self-Attention ব্যবহার করো, তুমি শুধু একটি দৃষ্টিকোণ থেকে summary তৈরি করতে পারবে। কিন্তু Multi-Head Attention দিয়ে একই document-

কে **multiple perspectives** থেকে দেখে multiple summaries তৈরি করা যায় — যা অনেক বেশি তথ্যবহুল।

SECTION 02



Self-Attention রিক্যাপ

Multi-Head Attention বোঝার আগে Self-Attention ভালো করে বুঝতে হবে। Self-Attention হলো এমন একটি technique যা দিয়ে **Contextual Embeddings** তৈরি করা যায়।

সমস্যাটা কী ছিল? — Static Embeddings-এর দুর্বলতা

⚠️ STATIC EMBEDDINGS-এর সমস্যা

"Money" এবং "Bank" শব্দের embedding সবসময় একই থাকে। কিন্তু:

- **"Money Bank"** — এখানে "Bank" মানে আর্থিক প্রতিষ্ঠান
- **"River Bank"** — এখানে "Bank" মানে নদীর তীর

Static embedding এই পার্থক্য ধরতে পারে না!

Self-Attention কীভাবে এই সমস্যা সমাধান করে?

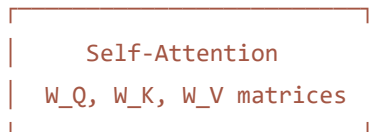
Self-Attention **Contextual Embeddings** তৈরি করে — অর্থাৎ একই শব্দ বিভিন্ন context-এ বিভিন্ন embedding পাবে।

Self-Attention প্রক্রিয়ার সংক্ষিপ্ত চিত্র:

Input Sentence: ["Money", "Bank"]



Static Embeddings: [E_money (dim=4), E_bank (dim=4)]



Contextual Embeddings: [Y_money, Y_bank]

(এখন "Bank"-এর embedding context অনুযায়ী পরিবর্তিত হয়)

Self-Attention-এর ভেতরে ঠিক কী হয়?

১

তিনটি Weight Matrix তৈরি করো

W_Q (Query), W_K (Key), W_V (Value) — এই তিনটি learnable matrix ব্যবহার করা হয়

২

প্রতিটি শব্দের জন্য Q, K, V Vector বের করো

প্রতিটি embedding-কে W_Q , W_K , W_V -এর সাথে Matrix Multiplication করো → প্রতিটি শব্দের জন্য তিনটি নতুন vector পাবে

৩

Similarity Score বের করো

প্রতিটি শব্দের Q vector-কে অন্য শব্দের K vector-এর সাথে Dot Product করো

৪

Scale করো

Score-গুলো $\sqrt{d_k}$ দিয়ে ভাগ করো — gradient flow ঠিক রাখতে

৫

Softmax দিয়ে Normalize করো

Scaled score-গুলোকে Softmax দিয়ে 0-1 এর মধ্যে নিয়ে আসো → এগুলো হলো Attention Weights

৬

Weighted Sum — Contextual Embedding তৈরি করো

Attention Weights দিয়ে V vectors-এর Weighted Sum করো → এটাই হলো Contextual Embedding (Z)

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$$

SECTION 03



Self-Attention-এর সীমাবদ্ধতা

🔥 সবচেয়ে বড় সমস্যা

Self-Attention একটি বাক্য বা document-কে শুধুমাত্র **একটি দৃষ্টিকোণ (Single Perspective)** থেকে দেখতে পারে। যদি একাধিক অর্থ বা দৃষ্টিকোণ থাকে, Self-Attention সেগুলো ধরতে পারে না।

উদাহরণ দিয়ে বোঝা যাক

বাক্যটি: **"The man saw the astronomer with a telescope"**

● সম্ভাব্য অর্থ ১

"Man" এবং "Telescope"-এর মধ্যে strong similarity
→ লোকটি টেলিস্কোপ দিয়ে দেখছে

● সম্ভাব্য অর্থ ২

"Astronomer" এবং "Telescope"-এর মধ্যে strong similarity
→ জ্যোতির্বিদের কাছে টেলিস্কোপ আছে

Self-Attention এই দুটোর মধ্যে যেকোনো **একটি** অর্থই ধরতে পারে — দুটো একসাথে নয়। এটাই মূল সমস্যা।

Self-Attention যা করতে পারে না

- একটি বাক্যের multiple interpretations একসাথে ধরা
- Syntactic relationship (ব্যাকরণিক সম্পর্ক) এবং Semantic relationship (অর্থগত সম্পর্ক) একসাথে capture করা
- Short-range এবং Long-range dependencies একই সাথে মনোযোগ দেওয়া
- বিভিন্ন ধরনের linguistic patterns একই সাথে শেখা

SECTION 04

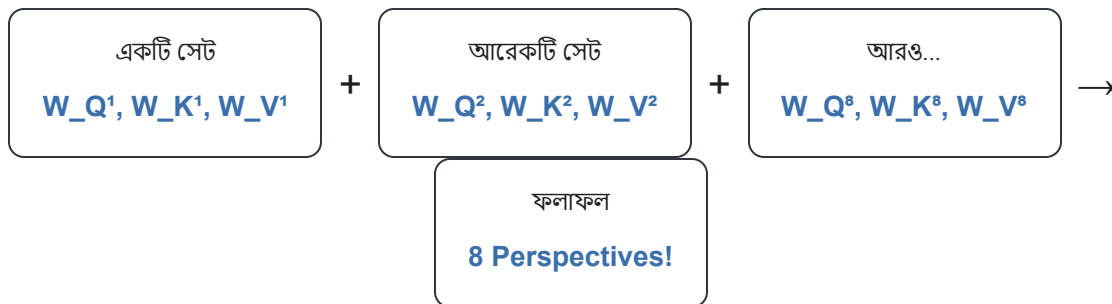


Multi-Head Attention — মূল সমাধান

✓ সমাধান কী?

একটি Self-Attention-এর বদলে **একাধিক Self-Attention মডিউল** একসাথে parallel-এ চালাও। প্রতিটি Self-Attention মডিউলকে বলা হয় একটি **"Head"**। একাধিক Head থাকলে তাকে বলা হয় **Multi-Head Attention**।

মূল আইডিয়াটা সহজ ভাষায়



প্রতিটি set of weights আলাদা আলাদাভাবে train হয়, তাই প্রতিটি head আলাদা pattern শেখে। এইভাবে **multiple perspectives** capture করা সম্ভব হয়।

✦ মনে রাখার বিষয়

Original "Attention Is All You Need" paper-এ **৪টি Attention Head** ব্যবহার করা হয়েছে — মানে একসাথে ৪টি Self-Attention মডিউল parallel-এ চলে, যা ৪টি ভিন্ন দৃষ্টিকোণ capture করে।

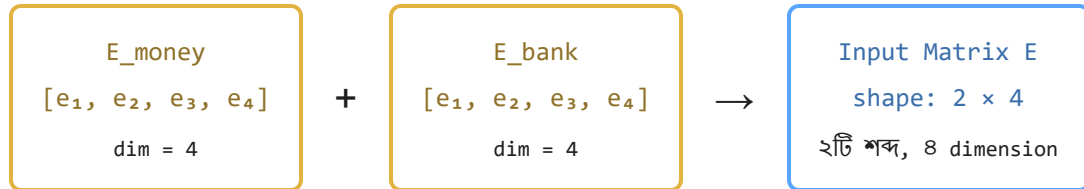
SECTION 05



ধাপে ধাপে সম্পূর্ণ প্রক্রিয়া (Matrix Level)

উদাহরণ হিসেবে নেওয়া হচ্ছে: **"Money Bank"** — ২টি শব্দ, প্রতিটির embedding dimension = 4, এবং আমরা 2টি Head ব্যবহার করব।

ধাপ ১ — Input Embeddings তৈরি করো



ধাপ ২ — Multiple Weight Matrices তৈরি করো (প্রতি Head-এর জন্য)

Head 1 এর জন্য:

W_Q^1 (4×4) W_K^1 (4×4) W_V^1 (4×4)

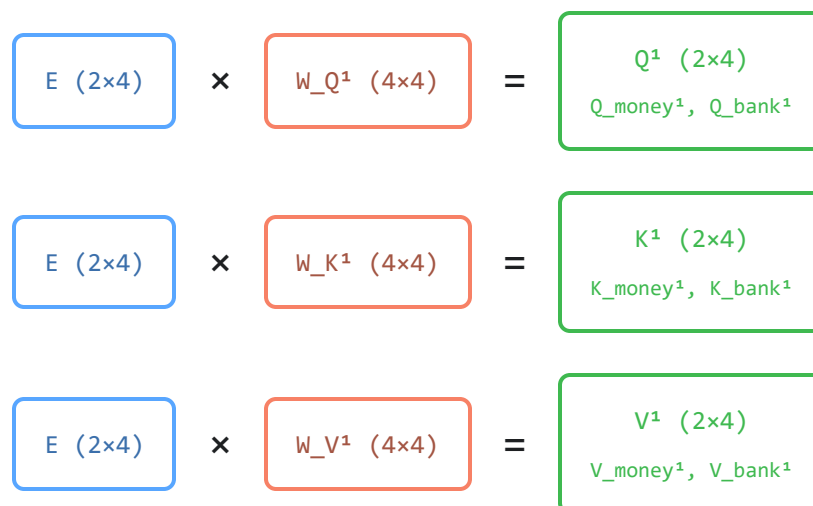
Head 2 এর জন্য:

W_Q^2 (4×4) W_K^2 (4×4) W_V^2 (4×4)

মোট Weight Matrices = 2 heads × 3 matrices = 6টি

এই সব weights training-এ শেখা হয়।

ধাপ ৩ — Q, K, V Matrices বের করো (প্রতি Head-এর জন্য)



একইভাবে Head 2 এর জন্যও Q^2 , K^2 , V^2 বের হবে। মোট এখন আমাদের কাছে আছে:

Q^1 K^1 V^1

Q^2 K^2 V^2

ধাপ ৪ — প্রতিটি Head-এ Self-Attention চালাও (Parallel)

Head 1:

Input: Q^1 , K^1 , V^1

Process: $\text{softmax}(Q^1 \times (K^1)^T / \sqrt{4}) \times V^1$

Output: $Z^1 = [Z_money^1, Z_bank^1]$ shape: 2×4

Head 2:

Input: Q^2 , K^2 , V^2

Process: $\text{softmax}(Q^2 \times (K^2)^T / \sqrt{4}) \times V^2$

Output: $Z^2 = [Z_money^2, Z_bank^2]$ shape: 2×4

দুটো Head একসাথে parallel-এ চলে।

Z^1 এবং Z^2 সম্পূর্ণ আলাদা – ভিন্ন perspective capture করে।

ধাপ ৫ — Concatenation করো (জোড়া লাগাও)

সব Head-এর output একসাথে জুড়ে দাও:

$$\begin{matrix} Z^1 \\ (2 \times 4) \end{matrix} \parallel \begin{matrix} Z^2 \\ (2 \times 4) \end{matrix} = \begin{matrix} Z_concat \\ (2 \times 8) \end{matrix}$$

SHAPE বিশ্লেষণ

Z^1 হলো 2×4 , Z^2 হলো $2 \times 4 \rightarrow$ Concatenate করলে হয় 2×8

(2টি শব্দ, প্রতিটির vector dimension এখন $8 = 4+4$)

ধাপ ৬ — Linear Transformation (W_output দিয়ে)

Concatenated output-এর dimension টি মূল embedding-এর dimension-এ ফিরিয়ে আনতে হবে। এজন্য একটি linear transformation matrix ব্যবহার করা হয়:

$$\begin{matrix} Z_concat \end{matrix} \times \begin{matrix} W_output \end{matrix} = \begin{matrix} Z_final \end{matrix}$$

(2×8) (8×4) (2×4)

✓ গুরুত্বপূর্ণ বিষয়

Final output-এর shape (2×4) = Input-এর shape (2×4) — একদম মিলে যাচ্ছে!

এই W_{output} matrix-এর weights-ও training-এ শেখা হয়। এটি বিভিন্ন perspective-গুলোর মধ্যে গুরুত্ব নির্ধারণ করে।

SECTION 06



Original Transformer Paper-এর Implementation

এখন দেখা যাক "Attention Is All You Need" paper-এ ঠিক কীভাবে Multi-Head Attention implement করা হয়েছে। মূল ধারণা একই, তবে কিছু গুরুত্বপূর্ণ পরিবর্তন আছে।

পরিবর্তন ১ — Embedding Dimension: ৫১২

EMBEDDING SIZE

Original paper-এ প্রতিটি শব্দের embedding dimension হলো **512**।

মানে E_money এবং E_bank উভয়ই 512-dimensional vector।

তাই Input Matrix E-এর shape হলো: **2 × 512**

পরিবর্তন ২ — ৮টি Attention Head

HEAD	W_Q MATRIX	W_K MATRIX	W_V MATRIX
Head 1	W_Q ¹ (512×64)	W_K ¹ (512×64)	W_V ¹ (512×64)
Head 2	W_Q ² (512×64)	W_K ² (512×64)	W_V ² (512×64)
...	...	...	...
Head 8	W_Q ⁸ (512×64)	W_K ⁸ (512×64)	W_V ⁸ (512×64)

পরিবর্তন ৩ — Dimension Reduction: ৫১২ থেকে ৬৪

🔑 কেন 512 থেকে 64-তে নামানো হয়?

উত্তর: Computational Efficiency (হিসাব কমানো)

যদি প্রতিটি Head 512 dimension নিয়ে কাজ করত, তাহলে ৮টি Head মিলে অনেক বেশি computation হতো।

কিন্তু প্রতিটি Head-কে 64 dimension দিলে: $8 \times 64 = 512$ — মোট computation একই থাকে!

অর্থাৎ: **512 dimension-এর একটি Self-Attention = 64 dimension-এর 8টি Head-এর Multi-Head Attention**

কিন্তু পাওয়া যাচ্ছে 8গুণ বেশি perspective!

Original Transformer Paper-এর সম্পূর্ণ প্রক্রিয়া:

Step 1: Input

$E = [E_money, E_bank]$ shape: 2×512

Step 2: Linear Projection (প্রতি head-এর জন্য)

$E (2 \times 512) \times W_{Q^1} (512 \times 64) = Q^1 (2 \times 64)$

$E (2 \times 512) \times W_{K^1} (512 \times 64) = K^1 (2 \times 64)$

$E (2 \times 512) \times W_{V^1} (512 \times 64) = V^1 (2 \times 64)$

... (8 heads-এর জন্য একইভাবে)

Step 3: Self-Attention (8 heads-এ parallel)

$Z^1 = \text{Attention}(Q^1, K^1, V^1)$ shape: 2×64

$Z^2 = \text{Attention}(Q^2, K^2, V^2)$ shape: 2×64

...

$Z^8 = \text{Attention}(Q^8, K^8, V^8)$ shape: 2×64

Step 4: Concatenation

$Z_concat = [Z^1 \parallel Z^2 \parallel \dots \parallel Z^8]$

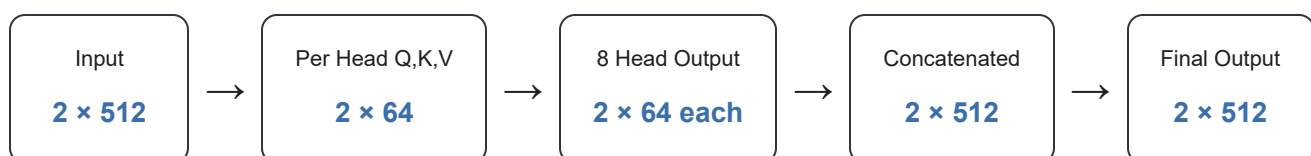
shape: $2 \times (64 \times 8) = 2 \times 512$

Step 5: Final Linear Transformation

$Z_concat (2 \times 512) \times W_{output} (512 \times 512) = Z_final (2 \times 512)$

Output: Z_final shape = 2×512 = Input shape ✓

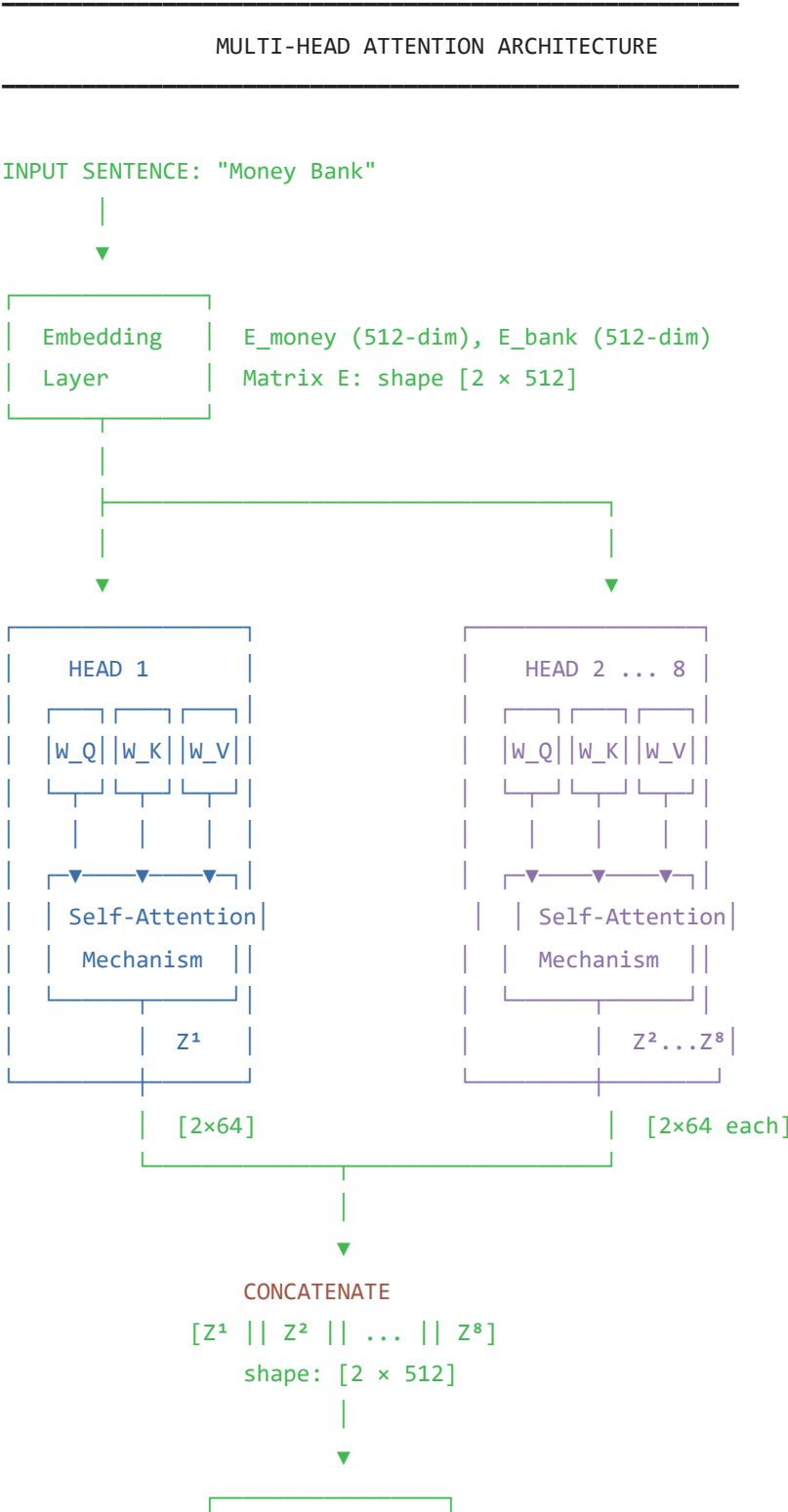
Shape Summary দেখা যাক:

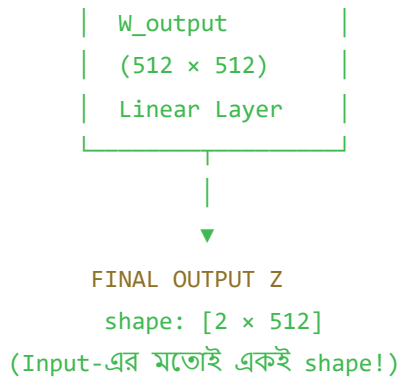




Visual Architecture Diagram

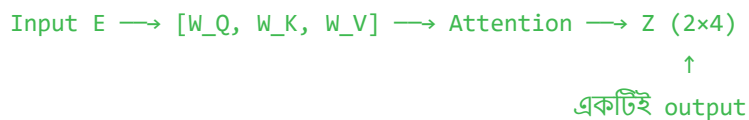
Multi-Head Attention-এর সম্পূর্ণ Architecture



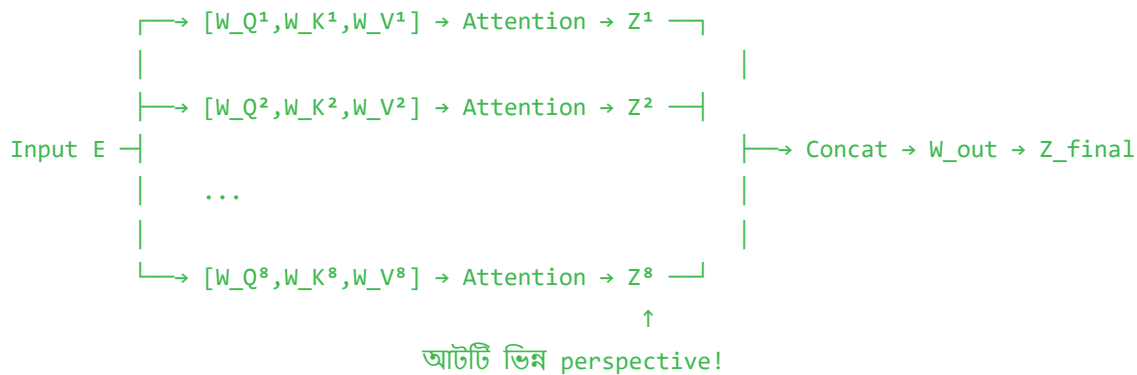


Self-Attention বনাম Multi-Head Attention — পার্থক্য

SELF-ATTENTION (Single Head):



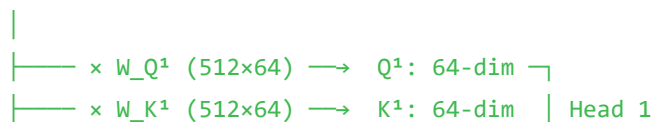
MULTI-HEAD ATTENTION (8 Heads):

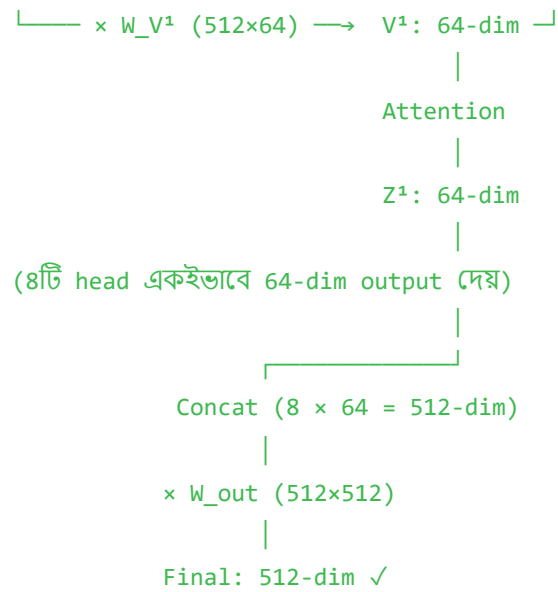


Dimension Flow Diagram

Embedding থেকে Final Output পর্যন্ত Dimension পরিবর্তন:

Word Embedding: 512-dim





SECTION 08



Visualization Tool Demo বিশ্লেষণ

Google-এর একটি visualization tool রয়েছে (googleapis.com) যেখানে Multi-Head Attention-এর কাজ চোখে দেখা যায়। এই tool-এ multiple layers-এ multiple heads দেখা যায়।

🔍 TOOL-এ কী দেখা যায়?

প্রতিটি শব্দের উপর hover করলে দেখা যায় সেই শব্দটি অন্য কোন শব্দের সাথে কতটুকু similar (attention weight কত)।

বাক্য: "The man saw the astronomer with a telescope"

● Head 0 (Layer 0) — প্রথম দৃষ্টিকোণ

যখন "man"-এ focus করা হয়:

- "man" → "telescope" সবচেয়ে strong similarity
- "man" → "saw" এবং "astronomer" দুর্বল

ধরা পড়ছে: লোকটি টেলিস্কোপ দিয়ে দেখছে

● Head 1 (Layer 0) — দ্বিতীয় দৃষ্টিকোণ

যখন "man"-এ focus করা হয়:

- "man" → "astronomer" সবচেয়ে strong
- "astronomer" → "telescope" সবচেয়ে strong

ধরা পড়ছে: জ্যোতির্বিদের কাছে টেলিস্কোপ আছে

💡 VISUALIZATION থেকে শিক্ষা

একই বাক্যে দুটো ভিন্ন Head দুটো ভিন্ন অর্থ capture করছে। Head 0 একটি interpretation ধরছে, Head 1 আরেকটি ধরছে। এভাবেই Multi-Head Attention বাক্যের সমৃদ্ধ অর্থ বোঝে।

SECTION 09



W_output Matrix-এর ভূমিকা

W_output বা Linear Transformation matrix শুধু dimension কমায় না — এটি আরও একটি গুরুত্বপূর্ণ কাজ করে।

কাজ ১ — Dimension ঠিক করা

$2 \times 512 \rightarrow 2 \times 512$ (same shape রাখা, অথবা প্রয়োজনমতো বদলানো)

কাজ ২ — Importance Weighting

বিভিন্ন head-এর output-এর গুরুত্ব নির্ধারণ করে — কোন perspective বেশি important?

কাজ ৩ — Perspective Mixing

বিভিন্ন perspective-এর একটি সমন্বিত (blended) representation তৈরি করে

কাজ ৪ — Training-এ Learnable

এই weights-ও backpropagation-এ শেখা হয় — তাই সময়ের সাথে optimize হয়

SECTION 10



সম্পূর্ণ Summary ও মূল বিষয়গুলো

Multi-Head Attention-এর পুরো প্রক্রিয়া এক নজরে

১

Input Embedding

বাক্যের প্রতিটি শব্দের Static Embedding নাও (word2vec বা অনুরূপ মডেল থেকে)

২

Multiple Weight Sets তৈরি করো

h সংখ্যক Head-এর জন্য h সেট W_Q, W_K, W_V matrix তৈরি করো (সব learnable)

৩

Linear Projection (Dimension Reduction)

প্রতিটি Head-এর জন্য: $\text{Embedding} \times W_Q/W_K/W_V \rightarrow \text{reduced dimension-এ } Q, K, V \text{ পাও}$

৪

Parallel Self-Attention

h সংখ্যক Self-Attention একসাথে parallel-এ চালাও $\rightarrow$ প্রতিটি থেকে একটি করে Contextual Output পাও

৫

Concatenation

সব Head-এর output জুড়ে দাও $\rightarrow$ একটি বড় combined matrix পাও

৬

Final Linear Transformation

W_output দিয়ে final dimension-এ আনো → এটাই Multi-Head Attention-এর চূড়ান্ত output

🏆 BEST OF BOTH WORLDS!

Multi-Head Attention ব্যবহার করলে:

- **Computation:** একটি 512-dim Self-Attention-এর সমান (একটু বেশি না)
- **Capability:** ৪টি ভিন্ন perspective capture করার ক্ষমতা
- **Learning:** প্রতিটি Head আলাদা linguistic pattern শেখে

অর্থাৎ তুমি একই computation খরচ করে অনেক বেশি তথ্য capture করতে পারছ!

Key Terms মনে রাখার জন্য

TERM	বাংলায় মানে	ROLE
Head	একটি Self-Attention মডিউল	একটি perspective capture করে
W_Q, W_K, W_V	Query, Key, Value Weight Matrix	Embedding থেকে Q/K/V বের করে
Q, K, V	Query, Key, Value Vector	Attention calculation-এ ব্যবহার হয়
Attention Weight	মনোযোগের মাত্রা	কোন শব্দ কতটুকু important সেটা বলে
Z (output)	Contextual Embedding	Context-aware representation
W_output	আউটপুট Weight Matrix	Heads-এর output mix করে
Concatenation	জুড়ে দেওয়া	সব head-এর output একত্রিত করা
d_k	Key Dimension	Scaling factor ($\sqrt{d_k}$ দিয়ে ভাগ)